

Hoare Logic – Lecture Notes

S P Suresh

Chennai Mathematical Institute

spsuresh@cmi.ac.in

1 Language syntax and semantics

1.1 Syntax

Arithmetic expressions are defined as follows:

$$e ::= n \mid X \mid e_1 + e_2 \mid e_1 - e_2 \mid e_1 \times e_2 \mid e_1 / e_2$$

where X, Y, Z and other uppercase letters stand for program identifiers, and $m, n, p, \dots$ and variants stand for natural numbers. The set of all identifiers is denoted by Id .

Boolean expressions are defined as follows:

$$b ::= e_1 = e_2 \mid e_1 \neq e_2 \mid e_1 \leq e_2 \mid e_1 \geq e_2 \mid \neg b_1 \mid b_1 \wedge b_2 \mid b_1 \vee b_2$$

where e_1, e_2 are arithmetic expressions.

Commands are defined as follows:

$$c ::= \text{skip} \mid X := e \mid c_1 ; c_2 \mid \text{if } b \text{ then } c_1 \text{ else } c_2 \text{ end} \mid \text{while } b \text{ do } c \text{ end}$$

1.2 Semantics

A *state* is a finite partial function s with $\text{dom}(s) \subseteq Id$ and $\text{rng}(s) \subseteq \mathbb{N}$. The meaning of an arithmetic expression e in state s , denoted $\llbracket e \rrbracket s$, is defined as below:

$$\begin{array}{llll} \llbracket n \rrbracket s & := & n & \llbracket e_1 + e_2 \rrbracket s & := & \llbracket e_1 \rrbracket s + \llbracket e_2 \rrbracket s & \llbracket e_1 \times e_2 \rrbracket s & := & \llbracket e_1 \rrbracket s \times \llbracket e_2 \rrbracket s \\ \llbracket X \rrbracket s & := & s(X) & \llbracket e_1 - e_2 \rrbracket s & := & \llbracket e_1 \rrbracket s - \llbracket e_2 \rrbracket s & \llbracket e_1 / e_2 \rrbracket s & := & \llbracket e_1 \rrbracket s \div \llbracket e_2 \rrbracket s \end{array}$$

The meaning of a boolean expression b in state s is defined as below:

$s \xrightarrow{\text{skip}} s$	$\frac{\llbracket e \rrbracket s = n}{s \xrightarrow{X := e} s[X \mapsto n]}$	$\frac{s \xrightarrow{c_1} s_1 \quad s_1 \xrightarrow{c_2} s_2}{s \xrightarrow{c_1 ; c_2} s_2}$
$\frac{s \models b \quad s \xrightarrow{c_1} s'}{s \xrightarrow{\text{IF}} s'}$	$\frac{s \not\models b \quad s \xrightarrow{c_2} s'}{s \xrightarrow{\text{IF}} s'}$	
$\frac{s \not\models b}{s \xrightarrow{\text{WHILE}} s}$	$\frac{s \models b \quad s \xrightarrow{c} s_1 \quad s_1 \xrightarrow{\text{WHILE}} s_2}{s \xrightarrow{\text{WHILE}} s_2}$	

Table 3: Big-step semantics. IF stands for **if** b **then** c_1 **else** c_2 **end**, and WHILE stands for **while** b **do** c **end**.

$$\begin{aligned}
s \models e_1 = e_2 & \text{ iff } \llbracket e_1 \rrbracket s = \llbracket e_2 \rrbracket s \\
s \models e_1 \neq e_2 & \text{ iff } \llbracket e_1 \rrbracket s \neq \llbracket e_2 \rrbracket s \\
s \models e_1 \leq e_2 & \text{ iff } \llbracket e_1 \rrbracket s \leq \llbracket e_2 \rrbracket s \\
s \models e_1 \geq e_2 & \text{ iff } \llbracket e_1 \rrbracket s \geq \llbracket e_2 \rrbracket s \\
s \models \neg b & \text{ iff } s \not\models b \\
s \models b_1 \wedge b_2 & \text{ iff } s \models b_1 \text{ and } s \models b_2 \\
s \models b_1 \vee b_2 & \text{ iff } s \models b_1 \text{ or } s \models b_2
\end{aligned}$$

The meaning of commands is given in terms of *big-step semantics*, defined in Table 3. It is “big-step” because we capture the state change effected by the entire command by the relation $s \xrightarrow{c} s'$, where c is a command and s, s' are states. In the definition, we use $s[X \mapsto n]$ to denote the state obtained by changing the meaning of X to n in s (and leaving it otherwise unchanged).

$$(s[X \mapsto n])(Y) = \begin{cases} n & \text{if } Y = X \\ s(Y) & \text{otherwise} \end{cases}$$

The following theorem can be proved by a straightforward induction.

THEOREM 1 (Determinism of commands). *For any command c and state s , there is at most one s' such that $s \xrightarrow{c} s'$.*

1.3 Example program and semantics

Consider the program in Figure 1, for computing the factorial of 3. Let c_1 and c_2 denote the commands $X := 3; Y := 1; I := 0$ and $I := I + 1; Y := Y \times I$. Let w denote **while** $I < X$ **do** c_2 **end**, and let

```

X := 3; Y := 1; I := 0;
while (I < X) do
  I := I + 1; Y := Y * I;
end

```

Figure 1: Program for computing the factorial of 3

$$\begin{array}{c}
\frac{000 \xrightarrow{[c_1]} 310}{000 \xrightarrow{[c]} 363} \\
\frac{310 \xrightarrow{[c_2]} 311}{310 \xrightarrow{[w]} 363} \\
\frac{311 \xrightarrow{[c_2]} 322}{311 \xrightarrow{[w]} 363} \\
\frac{322 \xrightarrow{[c_2]} 363}{322 \xrightarrow{[w]} 363} \\
\frac{363 \not\models I < X}{363 \xrightarrow{[w]} 363}
\end{array}$$

Figure 2: Tracing the program for factorial

c denote the whole program, i.e. $c = c_1 ; w$. Let (m, n, p) denote the state $[X \mapsto m, Y \mapsto n, I \mapsto p]$, further simplified to mnp when each of m, n and p is in single digits. We can show that $000 \xrightarrow{[c]} 363$, as in Figure 2.

We can even prove the following more general result.

THEOREM 2. For all $m \in \mathbb{N}$, $(0, 0, 0) \xrightarrow{[c]} (m, m!, m)$.

Proof. We first prove that for all $m, n, p \in \mathbb{N}$ with $p \leq m$, $(m, n, p) \xrightarrow{[w]} (m, f(m, n, p), m)$ where $f(m, n, p) := n \frac{m!}{p!}$. The statement about w is proved by induction on $m - p$.

For the base case, $m - p = 0$ means that $m = p$ and $f(m, n, p) = n$. Since $(m, n, p) \not\models I < X$, we have $(m, n, p) \xrightarrow{[w]} (m, n, p)$, i.e. $(m, n, p) \xrightarrow{[w]} (m, f(m, n, p), m)$.

For the case when $m - p > 0$, we have $p < m$ and hence $p + 1 \leq m$. We have the following equation:

$$f(m, n(p+1), p+1) = n(p+1) \frac{m!}{(p+1)!} = n(p+1) \frac{m!}{p! \cdot (p+1)} = n \frac{m!}{p!} = f(m, n, p).$$

By induction hypothesis (since $m - (p+1) < m - p$), we can assume that

$$(m, n(p+1), p+1) \xrightarrow{[w]} (m, f(m, n(p+1), p+1), m) = (m, f(m, n, p), m),$$

and so we can construct the following proof.

$$\frac{(m, n, p) \xrightarrow{[c_2]} (m, n(p+1), p+1) \quad \frac{\text{IH}}{(m, n(p+1), p+1) \xrightarrow{[w]} (m, f(m, n(p+1), p+1), m)}}{(m, n, p) \xrightarrow{[w]} (m, f(m, n, p), m)}$$

$\frac{}{\{\alpha\} \text{ skip } \{\alpha\}} \text{ Skip}$	$\frac{\models \alpha' \rightarrow \alpha \quad \{\alpha\} c \{\beta\} \quad \models \beta \rightarrow \beta'}{\{\alpha'\} c \{\beta'\}} \text{ Con}$
$\frac{}{\{\alpha(e)\} X := e \{\alpha(X)\}} \text{ Assign}$	$\frac{\{\alpha\} c \{\beta\} \quad \{\beta\} c' \{\varphi\}}{\{\alpha\} c ; c' \{\varphi\}} \text{ Seq}$
$\frac{\{\alpha \wedge b\} c \{\beta\} \quad \{\alpha \wedge \neg b\} c' \{\beta\}}{\{\alpha\} \text{ if } b \text{ then } c \text{ else } c' \text{ end } \{\beta\}} \text{ If}$	$\frac{\{b \wedge \iota\} c \{\iota\}}{\{\iota\} \text{ while } b \text{ do } c \text{ end } \{\iota \wedge \neg b\}} \text{ While}$

Table 4: Hoare logic rules

Observe that $(0, 0, 0) \xrightarrow{[c_1]} (m, 1, 0)$. Also $(m, 1, 0) \xrightarrow{[w]} (m, f(m, 1, 0), m)$. But

$$f(m, 1, 0) = 1 \cdot \frac{m!}{0!} = m!$$

and $c = c_1 ; w$, so $(0, 0, 0) \xrightarrow{[c]} (m, m!, m)$, as desired. $\dashv$

2 Hoare Logic

2.1 Hoare triples and rules of Hoare logic

Rather than reason directly with the transitions of a program, as above, we would like to reason abstractly about assertions that hold at the start of a program and at the end. This is captured by a *Hoare triple* $\{\alpha\} c \{\beta\}$, where c is a command, and α, β are first-order formulas over the language of arithmetic expressions and boolean expressions¹ which also contains program identifiers, apart from other quantified variables denoting natural numbers. α is referred to as the *precondition*, and β the *postcondition*, of the triple. The informal meaning of $\{\alpha\} c \{\beta\}$ is that whenever c is run starting from a state satisfying α , *if it terminates*, then the end state satisfies β . This is called a *partial correctness assertion*, since we do not demand that the command c terminates. Hoare logic gives us rules (presented in Table 4) to reason about these triples directly, based on the program structure. We say that $\vdash \{\alpha\} c \{\beta\}$ if there is a proof of the triple using the rules in Table 4.

The **Skip**, **Seq** and **If** rules are straightforward, but the other rules merit some discussion.

¹If our commands used a richer set of expressions, involving data types like floating-point numbers and characters, and other data structures like lists, arrays, etc., our assertion language will also be enriched correspondingly.

- In the **Assign** rule, $\alpha(X)$ is an assertion with the identifier X possibly occurring in it, and $\alpha(e)$ is the assertion obtained by replacing all occurrences (if any) of X in α by e . On the face of it, the rule seems wrong, and even stated in the reverse direction. But a few examples will help understand this better. If we want to ensure that some property α is satisfied by X after the assignment $X := 3$, it is natural to expect α to hold of 3 to begin with. Say $\alpha(X)$ asserts that X is odd. This is true after the assignment, because 3 was odd to begin with. Thus $\alpha(3)$ is an adequate precondition for $\alpha(X)$ in this case. The same logic works even when e contains program identifiers (including X). For instance, X is odd after the assignment $X := X + 1$ if $X + 1$ was odd before the assignment, i.e., X was even.
- In the **Con** rule, we have premises of the form $\models \alpha' \rightarrow \alpha$. This means that $s \models \alpha' \rightarrow \alpha$ for all states s . This is a semantic notion that is used in a proof system, which appears to be against the spirit of the formal proofs. We will discuss this in more detail in Section 2.6.
- The ι in the **While** rule is a *loop invariant*. It is a property that, if it is true at the beginning of a loop iteration, is re-established at the end of the iteration. Loop invariants play an important role in proving the correctness of programs, as we will see later.

2.2 Example proof

In the proof below, w and c_2 refer to the **while** loop and the body of the loop from the factorial example earlier, m is any natural number, and ι denotes the loop invariant $X = m \wedge I \leq X \wedge Y = I!$. We use (ASC) to mean that the conclusion is obtained by a combination of (some number of applications of) the **Assign**, **Seq** and **Con** rules. This is also indicated by the double lines in the proof. (SC) has a similar meaning.

$$\frac{\frac{\frac{}{\{X = m\} Y := 1; I := 0 \{ \iota \}}{\{X = m\} Y := 1; I := 0 \{ \iota \}} \text{ (ASC)} \quad \frac{\frac{\frac{}{\{ \iota \wedge I < X \} c_2 \{ \iota \}}{\{ \iota \} w \{ \iota \wedge \neg(I < X) \}} \text{ (ASC)} \quad \text{While}}{\{ \iota \} w \{ \iota \wedge \neg(I < X) \}} \text{ (SC)}}{\{X = m\} Y := 1; I := 0; w \{Y = m!\}} \text{ (SC)}$$

2.3 Soundness of the rules

We say that a Hoare triple $\{\alpha\} c \{\beta\}$ is *valid* (denoted $\models \{\alpha\} c \{\beta\}$) if the following holds:

For all states s, s' , if $s \models \alpha$ and $s \xrightarrow{[c]} s'$, then $s' \models \beta$.

THEOREM 3 (Soundness of Hoare Logic). *If $\vdash \{\alpha\} c \{\beta\}$ then $\models \{\alpha\} c \{\beta\}$.*

Proof. It suffices to show that the **Skip** and **Assign** rules (axioms) are valid, and that the other rules preserve validity. We only consider the **Assign**, **If** and **While** rules. The rest of the cases are easy to prove.

- For the **Assign** rule, consider any s, s' s.t. $s \models \alpha(e)$ and $s \xrightarrow{[X := e]} s'$. Then $s' = s[X \mapsto \llbracket e \rrbracket s]$. By pure logic, we have that $s \models \alpha[X := e]$ iff $s[X \mapsto \llbracket e \rrbracket s] \models \alpha$. Thus we see that $s' \models \alpha(X)$.
- For the **If** rule, consider any s, s' with $s \models \alpha$ and $s \xrightarrow{[\text{if } b \text{ then } c_1 \text{ else } c_2 \text{ end}]} s'$. There are two cases to consider – $s \models b$ and $s \xrightarrow{[c_1]} s'$, or $s \not\models b$ and $s \xrightarrow{[c_2]} s'$. In both cases we have $s' \models \beta$, since both $\{\alpha \wedge b\} c_1 \{\beta\}$ and $\{\alpha \wedge \neg b\} c_2 \{\beta\}$ are valid.
- For the **While** rule, we show that for any $n \in \mathbb{N}$, and any s, s' with $s \models \iota$ and a proof of $s \xrightarrow{[\text{while } b \text{ do } c \text{ end}]} s'$ of size $\leq n$, we have $s' \models \iota \wedge \neg b$.

Assume that the above statement holds for all $m < n$ and consider n, s, s' as above. Again there are two cases to consider. The first case is that $s \not\models b$ and $s = s'$, in which case $s' \models \iota \wedge \neg b$, as desired. The second case is that $s \models b$ and there is a state s'' such that $s \xrightarrow{[c]} s''$ and a proof of $s'' \xrightarrow{[\text{while } b \text{ do } c \text{ end}]} s'$ of size $m < n$. Since $\{\iota \wedge b\} c \{\iota\}$, we have $s'' \models \iota$. By IH, we then have $s' \models \iota \wedge \neg b$, as desired. $\dashv$

2.4 Gödel's β -function lemma

The next question to be addressed is the completeness of Hoare logic. A key ingredient is the classic result of Gödel presented below (where $\text{rm}(m, n)$ denotes the remainder obtained on dividing m by n).

LEMMA 4 (Gödel's β -function lemma). *Let $\beta(x, y, i)$ be the function $\text{rm}(x, 1 + (1 + i)y)$. Then for every $n \geq 0$ and every sequence $a_0, \dots, a_n$, there exist $u, v \in \mathbb{N}$ such that $a_i = \beta(u, v, i)$ for all $i \leq n$.*

Proof. Given $n \geq 0$ and $a_0, \dots, a_n$, choose $s \geq \max(n, a_0, \dots, a_n)$ and let $v = s!$. Now consider $v_0, \dots, v_n$ defined by $v_i = 1 + (1 + i)v$. We claim that v_i and v_j are relatively prime, for $0 \leq i < j \leq n$. For if not, some prime p that divides both v_i and v_j . Then we also have $p \mid v \times (j - i)$. Since p is prime, we have $p \mid (j - i)$ or $p \mid v$. Since $j - i < s$, we have $(j - i) \mid s!$, so $p \mid v$ in either case. From $p \mid v$ and $p \mid (1 + (1 + i)v)$, we get $p \mid 1$, a contradiction. Thus the v_i s are pairwise relatively prime.

Let $w = v_0 \times \dots \times v_n$, $A = \{0, \dots, w - 1\}$ and $B = \{(b_0, \dots, b_n) \mid \forall i \leq n : 0 \leq b_i < v_i\}$. Each element of B is an $(n + 1)$ -tuple whose i th component can take values from 0 to $v_i - 1$. The number of such tuples is $v_0 \times \dots \times v_n = w$. Thus $|A| = |B|$. Consider the function $f : A \rightarrow B$ given by

$$f(x) = (\text{rm}(x, v_0), \dots, \text{rm}(x, v_n)).$$

We claim that f is 1-1. Suppose $u \leq u' < e$ and $f(u) = f(u')$. This means that for all $i \leq n$, $\text{rm}(u, v_i) = \text{rm}(u', v_i)$. In other words, $v_i \mid (u' - u)$ for all i . Since the v_i s are pairwise relatively prime, we have $w \mid (u' - u)$. But $u \leq u' < w$, so $u' - u < w$ and so $w \mid (u' - u) \Rightarrow u = u'$. Thus we see that f is 1-1, and its domain and range are of same size, which means that f is also onto. In turn, this means that for any sequence $(b_0, \dots, b_n) \in B$, there is $u < w$ such that $\text{rm}(u, v_i) = b_i$ for all $i \leq n$. In particular, $(a_0, \dots, a_n) \in B$, since each $a_i \leq s < v < v_i$. Thus there exist u and v such that for all $i \leq n$, $a_i = \text{rm}(u, v_i) = \text{rm}(u, 1 + (1 + i)v) = \beta(u, v, i)$, as desired. $\dashv$

2.5 Completeness of the rules

The main theorem we prove in this section is the following.

THEOREM 5. *For every assertion ψ and command c , there is an assertion $\text{wlp}(c, \psi)$ ² with the following properties.*

1. *For all states s , $s \models \text{wlp}(c, \psi)$ iff for all states s' , if $s \xrightarrow{c} s'$ then $s' \models \psi$.*
2. $\vdash \{\text{wlp}(c, \psi)\} c \{\psi\}$.

Armed with the above, we can prove the completeness of Hoare logic.

THEOREM 6. *If $\models \{\varphi\} c \{\psi\}$ then $\vdash \{\varphi\} c \{\psi\}$.*

Proof. Suppose $\models \{\varphi\} c \{\psi\}$.

Consider any state s s.t. $s \models \varphi$. For all states s' s.t. $s \xrightarrow{c} s'$, we have $s' \models \psi$. By item 1 of Theorem 5, it follows that $s \models \text{wlp}(c, \psi)$. This shows that $\models \varphi \rightarrow \text{wlp}(c, \psi)$.

By item 2 of Theorem 5, we have $\vdash \{\text{wlp}(c, \psi)\} c \{\psi\}$, so $\vdash \{\varphi\} c \{\psi\}$, by the **Con** rule. $\dashv$

The rest of the section is devoted to the proof of Theorem 5. We start by defining $\text{wlp}(c, \psi)$ for all c and all ψ , by induction on the structure of c . The definition (and proofs of items 1 and 2) is straightforward except in the **while** case.

- $\text{wlp}(\mathbf{skip}, \psi) := \psi$.
 1. For any state s , $s \xrightarrow{\mathbf{skip}} s'$ iff $s = s'$, so $s \models \text{wlp}(\mathbf{skip}, \psi)$ iff $s' \models \psi$ for all s' s.t. $s \xrightarrow{\mathbf{skip}} s'$.
 2. $\vdash \{\psi\} \mathbf{skip} \{\psi\}$ by the **Skip** rule.
- $\text{wlp}(X := e, \psi(X)) := \psi(e)$.
 1. We have $s \xrightarrow{X := e} s'$ iff $s' = s[X \mapsto \llbracket e \rrbracket s]$, so $s \models \psi(e)$ iff $s[X \mapsto \llbracket e \rrbracket s] \models \psi(X)$ (by pure logic). Thus $s \models \psi(e)$ iff $s' \models \psi(X)$ for all s' s.t. $s \xrightarrow{X := e} s'$.

² wlp stands for “weakest liberal precondition”

2. $\vdash \{\psi(e)\} X := e \{\psi(X)\}$ by the **Assign** rule.
- $wlp(c_1; c_2, \psi) := wlp(c_1, wlp(c_2, \psi))$.
 1. Consider any state s . By IH applied on c_1 and c_2 , we have $s \models wlp(c_1, wlp(c_2, \psi))$ iff $s_1 \models wlp(c_2, \psi)$ for all s_1 s.t. $s \xrightarrow{c_1} s_1$, iff $s_2 \models \psi$ for all s_2 s.t. $s_1 \xrightarrow{c_2} s_2$. In other words, $s \models wlp(c_1; c_2, \psi)$ iff $s_2 \models \psi$ for all s_2 s.t. $s \xrightarrow{c_1; c_2} s_2$.
 2. By IH, $\vdash \{wlp(c_1, wlp(c_2, \psi))\} c_1 \{\psi\}$ and $\vdash \{wlp(c_2, \psi)\} c_2 \{\psi\}$. By the **Seq** rule, we have $\vdash \{wlp(c_1, wlp(c_2, \psi))\} c_1; c_2 \{\psi\}$.
 - $wlp(\text{if } b \text{ then } c_1 \text{ else } c_2 \text{ end}, \psi) := (b \rightarrow wlp(c_1, \psi)) \wedge (\neg b \rightarrow wlp(c_2, \psi))$.³ We use IF as shorthand for **if** b **then** c_1 **else** c_2 **end** in the following.
 1. Consider a state s . Suppose $s \models (b \rightarrow wlp(c_1, \psi)) \wedge (\neg b \rightarrow wlp(c_2, \psi))$, and let $s \xrightarrow{\text{IF}} s'$. Now either $s \models b$ or $s \models \neg b$. In the former case, $s \models wlp(c_1, \psi)$ and $s \xrightarrow{c_1} s'$. In the latter case, $s \models wlp(c_2, \psi)$ and $s \xrightarrow{c_2} s'$. So in either case, by IH (applied on c_1 or c_2 , as the case may be), $s' \models \psi$.
Conversely, suppose $s' \models \psi$ for all s' s.t. $s \xrightarrow{\text{IF}} s'$. If $s \models b$, we have $s' \models \psi$ for all s' s.t. $s \xrightarrow{c_1} s'$. By IH applied on c_1 , we have $s \models wlp(c_1, \psi)$. Similarly, if $s \models \neg b$, then $s \models wlp(c_2, \psi)$. Thus $s \models (b \rightarrow wlp(c_1, \psi)) \wedge (\neg b \rightarrow wlp(c_2, \psi))$.
 2. By IH, $\vdash \{wlp(c_i, \psi)\} c_i \{\psi\}$, for $i \in \{1, 2\}$. So we can construct the following proof. To save space, we use φ_b and $\varphi_{\neg b}$ to denote $(wlp(\text{IF}, \psi) \wedge b) \rightarrow wlp(c_1, \psi)$ and $(wlp(\text{IF}, \psi) \wedge \neg b) \rightarrow wlp(c_2, \psi)$, respectively.

$$\begin{array}{c}
\begin{array}{c} \text{IH} \\ \hline \vdash \varphi_b \quad \{wlp(c_1, \psi)\} c_1 \{\psi\} \\ \hline \{wlp(\text{IF}, \psi) \wedge b\} c_1 \{\psi\} \end{array} \quad \text{Con} \quad \begin{array}{c} \text{IH} \\ \hline \vdash \varphi_{\neg b} \quad \{wlp(c_2, \psi)\} c_2 \{\psi\} \\ \hline \{wlp(\text{IF}, \psi) \wedge \neg b\} c_2 \{\psi\} \end{array} \quad \text{Con} \\
\hline \{wlp(\text{IF}, \psi)\} \text{IF} \{\psi\} \quad \text{If}
\end{array}$$

We now tackle the **while** case. Suppose c is a command such that $wlp(c, \vartheta)$ is defined for all assertions ϑ , and consider the command **while** b **do** c **end** and the post-condition ψ . For ease of presentation, we assume, without loss of generality, that X and Y are the only program identifiers mentioned in b, c and ψ . (This set is anyway finite, and our proof can be adapted easily to the general case, at the cost of handling more indices.) We use $(e_1, e_1) = (e'_1, e'_2)$ as shorthand for $e_1 = e'_1 \wedge e_2 = e'_2$, for expressions e_1, e_2, e'_1, e'_2 . We also use **WHILE** as shorthand for **while** b **do** c **end**. For any state s and $m, n \in \mathbb{N}$, we use $s(m, n)$ to stand for $s[(X, Y) \mapsto (m, n)]$, the update of s by simultaneously changing the value of X to m and Y to n . For any formula φ , $\varphi(m, n)$ denotes the formula obtained

³We could equivalently have defined this as $(b \wedge wlp(c_1, \psi)) \vee (\neg b \wedge wlp(c_2, \psi))$.

by replacing all occurrences of X and Y by m and n , respectively. Similarly for a boolean expression b .

Our definition of wlp must satisfy the following.

$$\begin{aligned} s \models wlp(\text{WHILE}, \psi) \quad &\text{iff} \quad s' \models \psi \text{ for all } s' \text{ s.t. } s \xrightarrow{[\text{WHILE}]} s' \\ &\text{iff} \quad s_l \models \psi \text{ for all sequences } s_0, \dots, s_l \text{ s.t.} \\ &\quad s = s_0 \wedge \forall i < l : (s_i \models b \wedge s_i \xrightarrow{[c]} s_{i+1}) \wedge s_l \models \neg b. \end{aligned} \quad (1)$$

Since the only identifiers whose values change in the state transitions considered above are X and Y , we can rewrite Eq. 1 as follows, using a sequence of pairs.

$$\begin{aligned} s \models wlp(\text{WHILE}, \psi) \quad &\text{iff} \quad s(m_l, n_l) \models \psi \text{ for all sequences } (m_0, n_0), \dots, (m_l, n_l) \text{ s.t.} \\ &\quad s = s(m_0, n_0) \\ &\quad \wedge \forall i < l : (s(m_i, n_i) \models b \wedge s(m_i, n_i) \xrightarrow{[c]} s(m_{i+1}, n_{i+1})) \\ &\quad \wedge s(m_l, n_l) \models \neg b. \end{aligned} \quad (2)$$

Now $s(m_i, n_i) \models b$ iff $s \models b(m_i, n_i)$. Similarly, $s = s(m_0, n_0)$ iff $s \models (X, Y) = (m_0, n_0)$. Finally, if a state t differs from s only on the values it assigns to X and Y , then $t = s(m, n)$ iff $t \models (X, Y) = (m, n)$. So we can say the following:

$$\begin{aligned} s(m, n) \xrightarrow{[c]} s(m', n') \quad &\text{iff} \quad \text{there is } t \text{ s.t. } s(m, n) \xrightarrow{[c]} t \text{ and} \\ &\text{for all } t \text{ s.t. } s(m, n) \xrightarrow{[c]} t, t \models (X, Y) = (m', n'). \end{aligned} \quad (3)$$

The above can be rewritten in terms of wlp as follows (appealing to item 1 of Theorem 5 for c).

$$\begin{aligned} s(m, n) \xrightarrow{[c]} s(m', n') \quad &\text{iff} \quad s(m, n) \not\models wlp(c, 0 = 1) \text{ and} \\ &\quad s(m, n) \models wlp(c, (X, Y) = (m', n')). \end{aligned} \quad (4)$$

Since for any assertion ϑ we have $s(m, n) \models \vartheta$ iff $s \models \vartheta(m, n)$, we can recast Eq. 4 as follows.

$$s(m, n) \xrightarrow{[c]} s(m', n') \quad \text{iff} \quad s \models (\neg wlp(c, 0 = 1) \wedge wlp(c, (X, Y) = (m', n')))(m, n). \quad (5)$$

We can now recast Eq. 2 in the following form.

$$\begin{aligned} s \models wlp(\text{WHILE}, \psi) \quad &\text{iff} \quad s \models \forall l, (m_0, n_0), \dots, (m_l, n_l) : \{ \\ &\quad [(X, Y) = (m_0, n_0) \wedge \\ &\quad \forall i < l : (b \wedge \neg wlp(c, 0 = 1) \wedge wlp(c, (X, Y) = (m_{i+1}, n_{i+1}))) (m_i, n_i) \\ &\quad \wedge \neg b(m_l, n_l)] \rightarrow \psi(m_l, n_l) \}. \end{aligned} \quad (6)$$

We could take the RHS of the above as the definition of $wlp(\text{WHILE}, \psi)$, but the catch is that it is not an assertion, as there are unboundedly many quantifiers on the RHS. To remedy this, we appeal to Gödel's β -function lemma and encode the sequence $l, (m_0, n_0), \dots, (m_l, n_l)$ as two numbers u, v .

This leads to the official definition of $wlp(\text{WHILE}, \psi)$, where m_i and n_i are shorthand for $\beta(u, v, 2i + 1)$ and $\beta(u, v, 2i + 2)$ respectively:

$$\begin{aligned} \tau(u, v, l, x, y) &:= \beta(u, v, 0) = l \wedge (X, Y) = (m_0, n_0) \\ &\quad \wedge \forall i < l : (b \wedge \neg wlp(c, 0 = 1) \wedge wlp(c, (X, Y) = (m_i, n_i)) (m_{i+1}, n_{i+1}) \\ &\quad \wedge \neg b(\underline{m_l}) \wedge (m_l, n_l) = (x, y) \\ wlp(\text{WHILE}, \psi) &:= \forall u, v, l, x, y : [\tau(u, v, l, x, y) \rightarrow \psi(x, y)]. \end{aligned} \tag{7}$$

Item 1 of Theorem 5 is true for the above definition of $wlp(\text{WHILE}, \psi)$, since it was derived from the equivalence stated by item 1. To prove item 2 of Theorem 5, consider any s, s' s.t. $s \xrightarrow{[c]} s'$ and $s \models b \wedge wlp(\text{WHILE}, \psi)$. We shall show that $s' \models wlp(\text{WHILE}, \psi)$. Towards this, consider any $m, n \in \mathbb{N}$ and u, v, l s.t. $s' \models \tau(u, v, l, m, n)$. This means that there is s'' s.t. $s' \xrightarrow{[\text{WHILE}]} s''$ and $s'' \models (X, Y) = (m, n)$. Combining with $s \xrightarrow{[c]} s'$, we have $s \xrightarrow{[\text{WHILE}]} s'$, and combining with $s \models b$, we also have $s \models \tau(u', v', l + 1, m, n)$, where u' and v' encode the sequence of states from s to s'' . But $s \models \forall y, z, w, x, y : \tau(y, z, w, x, y) \rightarrow \psi(x, y)$. So $s \models \psi(m, n)$. It follows that $s' \models \psi(m, n)$.⁴ Thus we see that $s' \models \tau(u, v, l, m, n) \rightarrow \psi(m, n)$ for all $u, v, l, m, n \in \mathbb{N}$. In other words, $s' \models wlp(\text{WHILE}, \psi)$.

We have thus proved that $\models \{b \wedge wlp(\text{WHILE}, \psi)\} c \{wlp(\text{WHILE}, \psi)\}$. It follows that

$$\models (b \wedge wlp(\text{WHILE}, \psi)) \rightarrow wlp(c, wlp(\text{WHILE}, \psi)).$$

By IH (item 2 of Theorem 5 applied on c), we have that

$$\vdash \{wlp(c, wlp(\text{WHILE}, \psi))\} c \{wlp(\text{WHILE}, \psi)\}.$$

We can now obtain the following proof for **WHILE**, where we use φ_b and $\varphi_{\neg b}$ as shorthands for $(b \wedge wlp(\text{WHILE}, \psi)) \rightarrow wlp(c, wlp(\text{WHILE}, \psi))$ and $(wlp(\text{WHILE}, \psi) \wedge \neg b) \rightarrow \psi$ respectively.

$$\begin{array}{c} \text{IH} \\ \hline \vdash \varphi_b \quad \{wlp(c, wlp(\text{WHILE}, \psi))\} c \{wlp(\text{WHILE}, \psi)\} \\ \hline \{b \wedge wlp(\text{WHILE}, \psi)\} c \{wlp(\text{WHILE}, \psi)\} \quad \text{Con} \\ \hline \{wlp(\text{WHILE}, \psi)\} \text{WHILE } \{wlp(\text{WHILE}, \psi) \wedge \neg b\} \quad \text{While} \\ \hline \{wlp(\text{WHILE}, \psi)\} \text{WHILE } \{\psi\} \quad \vdash \varphi_{\neg b} \quad \text{Con} \end{array}$$

2.6 A note on the **Con** rule – more Gödel

Note that the **Con** rule has premises of the form $\models \alpha' \rightarrow \alpha$, meaning that $s \models \alpha' \rightarrow \alpha$ for all states s . This notion of validity can be reduced to the truth of a *pure assertion* (one that does not contain any identifiers), as follows. Consider an assertion γ , and let X, Y, Z be the only identifiers occurring

⁴Since $\psi(\underline{m})$ has no program identifiers occurring in it, it is true in s iff it is true in s' .

in γ . For any state s and $m, n, p \in \mathbb{N}$, if $s = [X, Y, Z \mapsto m, n, p]$, then $s \models \gamma$ iff $\gamma(m, n, p)$ is true (over the standard model $\mathbb{N}$ for arithmetic). For every $m, n, p \in \mathbb{N}$, we have a state s that maps X, Y, Z to m, n, p . So instead of quantifying over states, we can quantify over numbers. Thus $\models \gamma$ iff $\mathbb{N} \models \forall x, y, z : \gamma(x, y, z)$. So all uses of $\models \gamma$ in any Hoare logic proof can be replaced by $\mathbb{N} \models \mu$ for an appropriate assertion μ which is pure.

This is still a semantic notion, and we can get a formal proof system only if we can replace each $\mathbb{N} \models \mu$ with $T \vdash_{\text{FOL}} \mu$ (for an appropriate recursive set of axioms T). But by Gödel's First Incompleteness Theorem, this is impossible, so we cannot hope to have a formal proof system that proves all valid Hoare triples! In practice, we usually work with some recursively axiomatizable fragment of arithmetic (like PA), which would actually provide formal proofs for a (considerably large) set of valid Hoare triples, though not all. By strengthening the axiomatization for the underlying arithmetic, one can prove more triples. Thus it is appropriate to say that Hoare logic is *relatively complete*, given the dependence on the underlying axiomatization for arithmetic.

3 Verification conditions

We saw the importance of weakest liberal preconditions in proving the completeness of Hoare logic. Completeness says that any valid Hoare triple can be proved using the rules. But while reasoning about programs, it is impractical to compute the *wlp* of **while** loops. It is customary to verify programs that come with appropriate *annotations*. These annotations are like **assert** statements inserted in programs. Rather than pepper the program with these **assert**s everywhere, we only assume that every loop is provided with an invariant (apart from the precondition and postcondition of the overall program). Formally, the syntax of annotated commands is given below, where ι is an assertion.

$$c ::= \text{skip} \mid X := e \mid c_1 ; c_2 \mid \text{if } b \text{ then } c_1 \text{ else } c_2 \text{ end} \mid \text{while } b \text{ inv } \iota \text{ do } c \text{ end}$$

Checking the Hoare triple $\{\alpha\} \text{ while } b \text{ inv } \iota \text{ do } c \text{ end} \{\beta\}$ is easy, in the presence of the purported invariant ι . One just checks the triple $\{b \wedge \iota\} c \{\iota\}$, and the implications $\alpha \rightarrow \iota$ and $\iota \wedge \neg b \rightarrow \beta$.

In fact, once we have loop invariants, validity of Hoare triples reduces to validity of assertions. We already know that $\models \{\varphi\} c \{\psi\}$ iff $\mathbb{N} \models \forall \underline{x} : (\varphi(\underline{x}) \rightarrow \text{wlp}(c, \psi)(\underline{x}))$. So all we need to do is compute *wlp* for annotated programs. For the other constructs, the definition is just as given earlier. But we can use the loop invariants to simplify the **while** case.

$$\begin{aligned} \text{wlp}(\text{while } b \text{ inv } \iota \text{ do } c \text{ end}, \psi) &:= \iota(\underline{x}) \wedge \\ &\quad \forall \underline{x} : ((\iota \wedge b) \rightarrow \text{wlp}(c, \iota)(\underline{x})) \wedge \\ &\quad \forall \underline{x} : ((\iota \wedge \neg b) \rightarrow \psi)(\underline{x}) \end{aligned}$$

The pure assertion encoding a Hoare triple is called a *verification condition*, and the above process is called verification condition generation, or VCGEN. One can now use standard techniques for proving these verification conditions, and even automate the proofs using tools like Coq.

An example annotated program is given below (in a C-like notation).

```

assert(n ≥ 0);
X := 0; Y := 0; Z := 1;
while (¬(X = n)) do
    assert(fib(X) = Y && fib(X+1) = Z);
    T := Z; Z := Y+Z; Y := T; X := X+1
end
assert(Y = fib(n));

```

One can verify that the following verification condition is generated, where we use $f(i)$ to denote the i -th Fibonacci number.

$$\begin{aligned}
 n \geq 0 \quad \rightarrow \quad & f(0) = 0 \wedge f(1) = 1 \wedge \\
 & \forall x, y, z : (f(x) = y \wedge f(x+1) = z \wedge x \neq n \rightarrow f(x+1) = z \wedge f(x+2) = y+z) \wedge \\
 & \forall x, y, z : (f(x) = y \wedge f(x+1) = z \wedge x = n \rightarrow y = f(n)).
 \end{aligned}$$